AI powered speech-to-speech mock interview platform

Noel Biju, Adwaith V, Faiha Safeer, Aisha Mol AA
Dept. of Computer Science & Engineering, College of Engineering Poonjar, Kottayam, Kerala, India
noelbijuk@gmail.com | *unnikuttan8t@gmail.com* | *faihasafeer43@gmail.com* | *aishamolaa2@gmail.com*

1. Team Roles Overview

Work is divided along the system's four natural pipeline layers, so each member owns one layer end-to-end while staying in sync with the others through shared API contracts defined early in the project.

Member	Role	Primary Ownership
Faiha Safeer (Member A)	Frontend & UX	Next.js web app: resume upload, interview screen, results/feedback screen, audio recording UI
Aisha Mol AA (Member B)	Backend & Orchestration	FastAPI service, session/state management, database schema, connecting all modules into one working pipeline
Adwaith V (Member C)	Speech Pipeline	Speech-to-text (faster-whisper) and text-to-speech (Piper) integration, audio pre/post-processing, latency optimization
Noel Biju (Member D)	AI / ML (Resume & Questions)	Resume parsing into structured data, prompt engineering for question generation, answer evaluation/scoring logic

2. Phase-by-Phase Task Allocation

Phase 1 — Foundations & Standalone Prototypes

Member	Task
A	*Set up Next.js project, build wireframes, build resume upload screen with mock data, basic interview screen layout.*
B	*Set up FastAPI skeleton, design database schema (sessions, transcripts, scores), define API contracts with the whole team.*
C	*Set up faster-whisper locally, test transcription accuracy on sample audio, set up Piper and test sample voice output.*
D	*Build resume text extraction (PyMuPDF/pdfplumber), test on 3-5 sample resumes, draft first prompt for structured extraction.*

Phase 2 — Core Integration

Member	Task
A	*Continue building UI screens against mock API responses; prepare audio recording component (MediaRecorder)*
B	*Connect D's resume parser and question generator into a single callable API; wire C's STT/TTS into backend services*
C	*Package STT/TTS as backend-callable services; begin latency testing and model size tuning*
D	*Finalize structured resume extraction; build the question generation prompt chain using resume JSON as input*

Phase 3 — Full Product Build

Member	Task
A	*Wire real UI to backend APIs; build full session flow (upload -> interview -> results); polish audio playback/recording UX*
B	*Handle session state across the full interview flow; ensure smooth handoff between question, answer, and evaluation steps*
C	*Optimize turnaround time end-to-end; add fallback handling for failed transcription or audio errors*
D	*Build the answer evaluation/scoring module (rubric-based); generate the final feedback report content*

Month 4 — Testing, Optimisation & Delivery

Member	Task
A	*UI polish based on test feedback; fix usability issues found during testing; prepare demo walkthrough*
B	*Bug fixes across the pipeline; ensure reliability under repeated test sessions; prepare deployment if applicable*
C	*Final latency tuning; verify STT/TTS reliability across different voices/ accents from test users*
D	*Refine question quality and scoring accuracy based on real test transcripts; tune prompts for consistency*

3. Shared Responsibilities

Some tasks don't belong to a single pipeline layer and are owned by the whole team together:

- Weekly sync meetings to review integration progress and blockers.
- Maintaining the shared GitHub repository, issue tracker, and API contract documentation.
- User testing sessions with real students and collecting feedback.
- Final project report, presentation slides, and demo rehearsal.
- Deciding on and documenting any scope cuts if the team falls behind schedule.

4. Escalation & Risk Ownership

If a specific layer falls behind schedule, the following applies to keep the whole project on track rather than letting one delay block everyone:

- If C (speech pipeline) is delayed, B should temporarily wire in a simpler placeholder (e.g. text only fallback) so integration testing can continue on other fronts.

- If D (AI/ML) is delayed, B can temporarily use a fixed, hardcoded question set so A and C's work isn't blocked.

- If B (backend) is delayed, this is the highest-priority blocker for the whole team, since every other module depends on it — all members should be ready to assist B directly if this happens.

- Any member significantly ahead of schedule should support B during the Phase 2-integration phase, since that is the project's highest-risk period.